The University of Azad Jammu and Kashmir, Muzaffarabad



Department of Software Engineering
Digital Image Processing Lab

Lab Task 03: Image Enhancement and Intensity Transformations

Name	Jawahir Ali
Roll No	2024-SE-34
Session / Semester	2024-28 (SE), Fifth Semester
Lab Instructor	Engr. Fahad Nisar
Assigned / Due	November 02, 2026 / November 09, 2026, 11:59 PM

1. Objective

To calculate, implement and compare grayscale intensity transformations (brightness, contrast, negative, gamma), histogram enhancement (global equalization and CLAHE) and thresholding (fixed, Otsu and adaptive Gaussian), while preventing overflow and interpreting the effect of each operation.

2. Input Image



Properties printed by the notebook:

shape = (800,800), dtype = uint8, min = 0, max = 255.

The image was loaded as 8-bit grayscale and kept unchanged (set read-only); every operation creates a new array. Arithmetic is done in int32/float64, the result is clipped to 0–255 and only then converted to uint8, which truncates any fraction. All grayscale figures use vmin = 0 and vmax = 255.



3. Dry Runs, Results and Observations

Each dry run below shows the hand calculation (raw values, clipping, final values). Every one of them was also verified in Python in the notebook.

Dry Run 1: Brightness adjustment

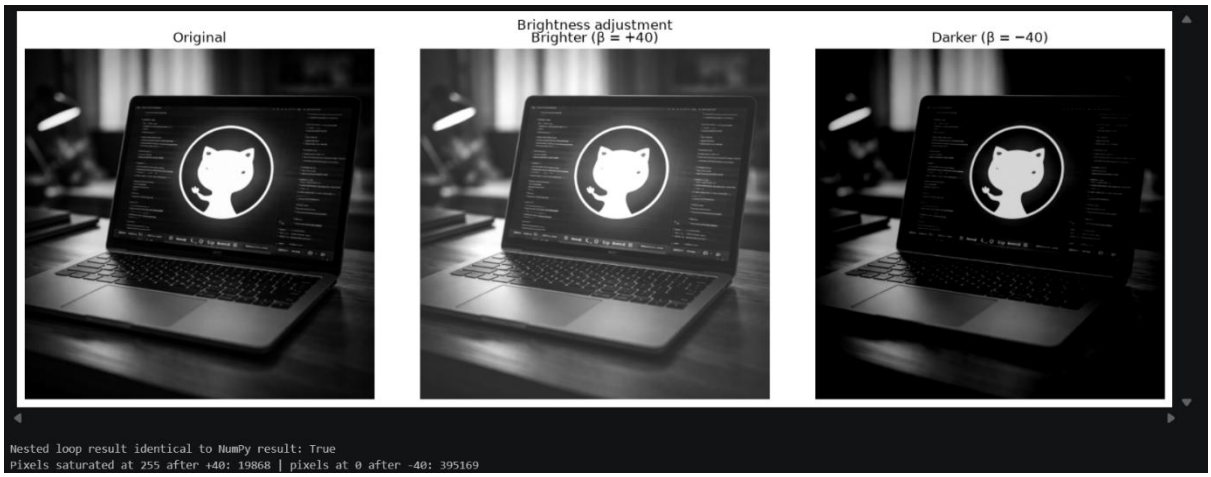
Formula: s = r + β, with β = +40 and β = −40. The sum is computed in a wider type (int), clipped to 0–255, then converted to uint8.

r	r+40 (raw)	Clipped	uint8	r−40 (raw)	Clipped	uint8
30	70	70	70	-10	0	0
80	120	120	120	40	40	40
128	168	168	168	88	88	88
180	220	220	220	140	140	140
240	280	255	255	200	200	200

Lost detail: 240 + 40 = 280 exceeds 255, so it is clipped to 255; 30 − 40 = −10 falls below 0, so it is clipped to 0. After clipping, every input ≥ 215 becomes 255 with β = +40, and every input ≤ 40 becomes 0 with β = −40. Different original values merge into the same output value, and this detail cannot be recovered. Without clipping, uint8 arithmetic would wrap around (280 → 24, −10 → 246), which would turn bright pixels dark and vice versa.

Python verification: np.clip(r.astype(int) + 40, 0, 255).astype(np.uint8) gives [70, 120, 168, 220, 255]; with −40 it gives [0, 40, 88, 140, 200]. Both match the table.

Notebook output:



Observation: Adding +40 shifts the whole histogram right and clips values ≥ 215 to 255; −40 shifts it left and clips values ≤ 40 to 0. The nested-loop result matched the NumPy result exactly.

Dry Run 2: Linear contrast from zero (α = 1.5, β = 0)

Formula: s = 1.5·r. Clip to 0–255, then truncate the fractional part.

r	1.5·r	Clipped	Truncated uint8	Saturated?
30	45	45	45	No
80	120	120	120	No
128	192	192	192	No
180	270	255	255	Yes
240	360	255	255	Yes

Saturated pixels: 180 (→ 270) and 240 (→ 360) both become 255, so highlights are clipped and their difference is lost. Dark pixels (30 → 45, 80 → 120) are only mildly brightened, and the whole image becomes brighter, not just higher in contrast, because the scaling is centered at 0.

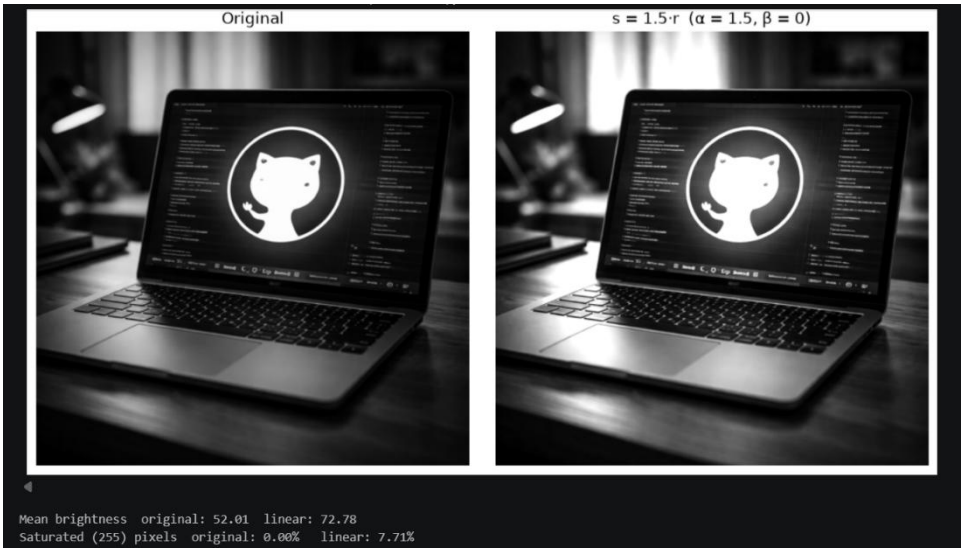
Notebook output:

Dry run 2 — pixels [30, 80, 128, 180, 240]

```
P = np.array([30, 80, 128, 180, 240])
raw = 1.5 * P
final = to_u8(raw)
for r, a, b in zip(P, raw, final):
    print(f"r={r:3d}  1.5r={a:6.1f}  clipped+truncated={b:3d}  saturated={a > 255}")
assert final.tolist() == [45, 120, 192, 255, 255]
```

✓ 0.0s

```
r= 30  1.5r=  45.0  clipped+truncated= 45  saturated=False
r= 80  1.5r= 120.0  clipped+truncated=120  saturated=False
r=128  1.5r= 192.0  clipped+truncated=192  saturated=False
r=180  1.5r= 270.0  clipped+truncated=255  saturated=True
r=240  1.5r= 360.0  clipped+truncated=255  saturated=True
```



Observation: Multiplying by 1.5 from zero makes the image brighter overall and saturates every pixel with r ≥ 170, so highlight detail is lost.

Dry Run 3: Contrast centered at 128

Formula: s = (r − 128)·α + 128, with α = 1.5 and α = 0.5.

α = 1.5

r	r − 128	×1.5	+128 (raw)	Clipped	Truncated uint8
30	-98	-147	-19	0	0
80	-48	-72	56	56	56
128	0	0	128	128	128
180	52	78	206	206	206
240	112	168	296	255	255

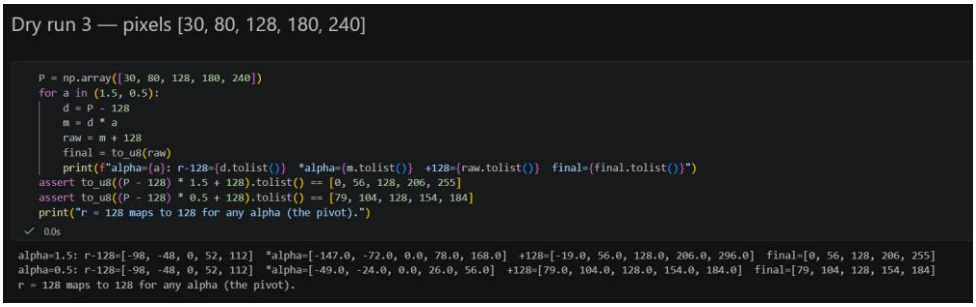
α = 0.5

r	r − 128	×0.5	+128 (raw)	Clipped	Truncated uint8
30	-98	-49	79	79	79
80	-48	-24	104	104	104
128	0	0	128	128	128
180	52	26	154	154	154
240	112	56	184	184	184

Result at r = 128: (128 − 128)·α + 128 = 128 for every α. The value 128 is the pivot (fixed point) of this transform, so mid-gray never changes. With α = 1.5 values move away from 128 (30 → 0 after clipping and 240 → 255 after clipping), so contrast increases; with α = 0.5 they move toward 128 (30 → 79, 240 → 184), so contrast decreases.

Comparison with Dry Run 2: s = 1.5r also stretches the values but pushes the whole range upward (mean brightness increases and highlights saturate), while the centered version keeps the middle tone fixed and stretches both dark and bright sides symmetrically.

Notebook output:





```
original      mean= 52.01 std= 60.47 pct@0= 2.52% pct@255= 0.00%
1.5r          mean= 72.78 std= 77.05 pct@0= 2.52% pct@255= 7.71%
centered 1.5  mean= 36.81 std= 70.33 pct@0=63.75% pct@255= 3.24%
centered 0.5  mean= 89.76 std= 30.24 pct@0= 0.00% pct@255= 0.00%
```

Observation: Centering at 128 keeps mid-gray fixed: α = 1.5 increases contrast on both sides (some clipping), α = 0.5 reduces contrast. Compared with 1.5r it does not raise overall brightness.

Dry Run 4: Image negative

r	s = 255 − r	Negative again: 255 − s	Equals original?
30	225	30	Yes
80	175	80	Yes
128	127	128	Yes
180	75	180	Yes
240	15	240	Yes

Applying the negative twice returns the original exactly, because 255 − (255 − r) = r. The transform is its own inverse and loses no information.

Notebook output:

```
P = np.array([30, 80, 128, 180, 240])
neg = 255 - P
neg2 = 255 - neg
print("negative      :", neg.tolist())
print("negative twice:", neg2.tolist(), "| equals original:", np.array_equal(neg2, P))
assert neg.tolist() == [225, 175, 127, 75, 15] and np.array_equal(neg2, P)
```

✓ 0.0s

```
negative      : [225, 175, 127, 75, 15]
negative twice: [30, 80, 128, 180, 240] | equals original: True
```

```
negative = to_u8(255 - original.astype(np.int32))
show_pair(original, "Original", negative, "Negative (s = 255 − r)")
print("Negative of negative equals original:", np.array_equal(to_u8(255 - negative.astype(np.int32)), original))
```

✓ 0.9s



Observation: The negative inverts all intensities and applying it twice restores the original. It helps inspection when faint dark detail on a dark background (e.g. X-ray structures) must be seen.

Dry Run 5: Gamma correction

Formula: s = 255·(r/255)^γ. Full precision is kept in every step; only the final result is truncated.

γ = 0.5

r	Normalized r/255	Power ^0.5	×255	Truncated uint8
0	0	0	0	0
64	0.25098	0.500979	127.7498	127
128	0.501961	0.708492	180.6654	180
192	0.752941	0.867722	221.2691	221
255	1	1	255	255

γ = 2.0

r	Normalized r/255	Power ^2.0	×255	Truncated uint8
0	0	0	0	0
64	0.25098	0.062991	16.0627	16
128	0.501961	0.251965	64.251	64
192	0.752941	0.56692	144.5647	144
255	1	1	255	255

Observation: with γ = 0.5 the dark and mid values are lifted (64 → 127, 128 → 180), so dark regions become brighter. With γ = 2.0 they are pushed down (64 → 16, 128 → 64), so the image becomes darker. The end points stay fixed in both cases (0 → 0 and 255 → 255) because 0^γ = 0 and 1^γ = 1.

Difference from the rounded slide example: a slide that rounds the normalized value first (e.g. 64/255 ≈ 0.25, √0.25 = 0.5, 0.5 × 255 = 127.5 ≈ 128) can show a value that is 1 higher than the full-precision computation. With full precision, 64/255 = 0.250980…, √ = 0.501975…, × 255 = 127.7498, and truncation gives 127. Early rounding and rounding versus truncation account for the 1-level difference.

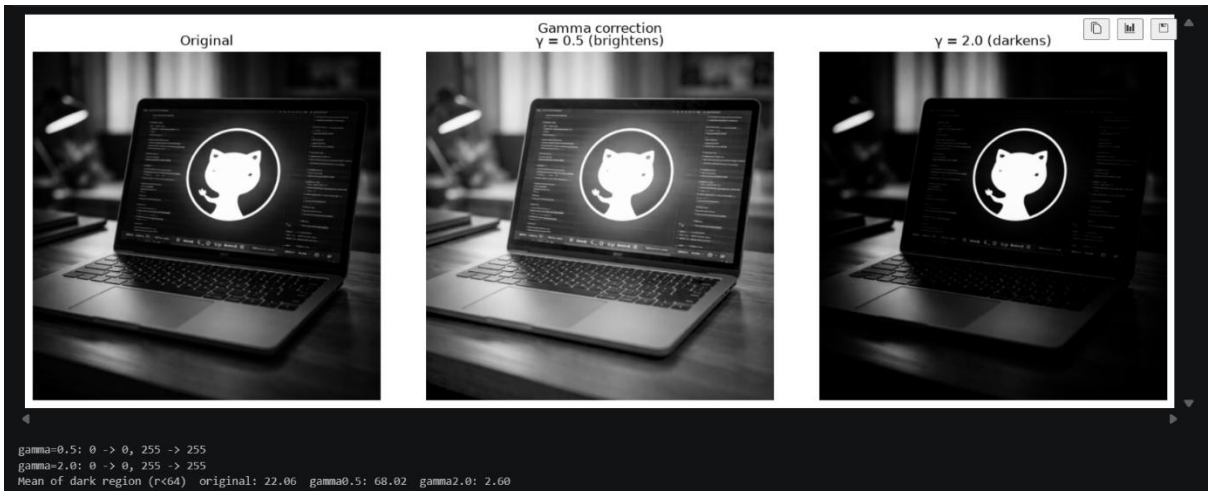
Notebook output:

Dry run 5 — pixels [0, 64, 128, 192, 255]

```
G = np.array([0, 64, 128, 192, 255])
for g in (0.5, 2.0):
    n = G / 255.0
    pw = n ** g
    sc = 255 * pw
    fin = to_u8(sc)      # full precision kept, truncate only at the end
    print(f"gamma={g}")
    for r, a, b, c, d in zip(G, n, pw, sc, fin):
        print(f"   r={r:3d} norm={a:.6f} power={b:.6f} x255={c:9.4f} final={d}")
assert to_u8(255 * (G / 255.0) ** 0.5).tolist() == [0, 127, 180, 221, 255]
assert to_u8(255 * (G / 255.0) ** 2.0).tolist() == [0, 16, 64, 144, 255]
```

✓ 0.0s

```
gamma=0.5
   r=  0 norm=0.000000 power=0.000000 x255=   0.0000 final=0
   r= 64 norm=0.250980 power=0.500979 x255= 127.7498 final=127
   r=128 norm=0.501961 power=0.708492 x255= 180.6654 final=180
   r=192 norm=0.752941 power=0.867722 x255= 221.2691 final=221
   r=255 norm=1.000000 power=1.000000 x255= 255.0000 final=255
gamma=2.0
   r=  0 norm=0.000000 power=0.000000 x255=   0.0000 final=0
   r= 64 norm=0.250980 power=0.062991 x255=  16.0627 final=16
   r=128 norm=0.501961 power=0.251965 x255=  64.2510 final=64
   r=192 norm=0.752941 power=0.566920 x255= 144.5647 final=144
   r=255 norm=1.000000 power=1.000000 x255= 255.0000 final=255
```



Observation: γ = 0.5 brightens and expands dark regions, γ = 2.0 darkens and compresses them. 0 and 255 remain fixed for every γ.

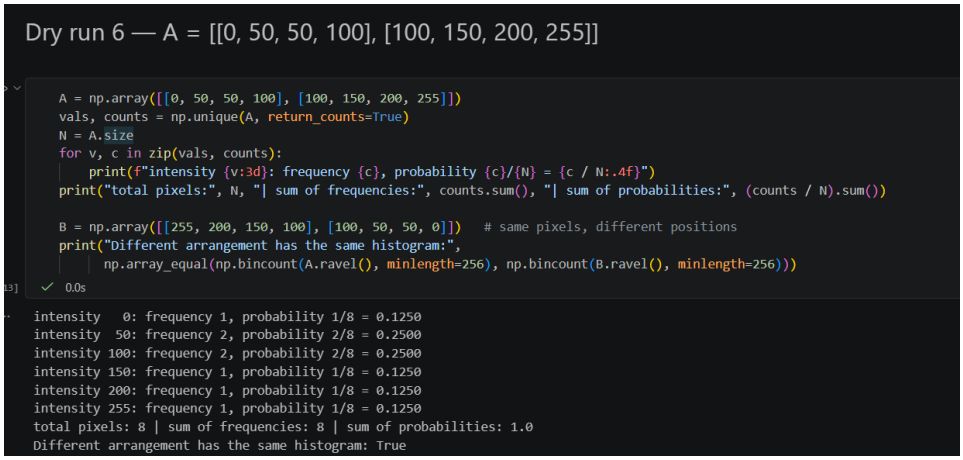
Dry Run 6: Grayscale histogram

Intensity	Frequency	Probability	Decimal
0	1	1/8	0.125
50	2	2/8	0.25
100	2	2/8	0.25
150	1	1/8	0.125
200	1	1/8	0.125
255	1	1/8	0.125

Total pixel count = 8 (the frequencies sum to 8 and the probabilities sum to 1). All other 250 intensity bins have frequency 0.

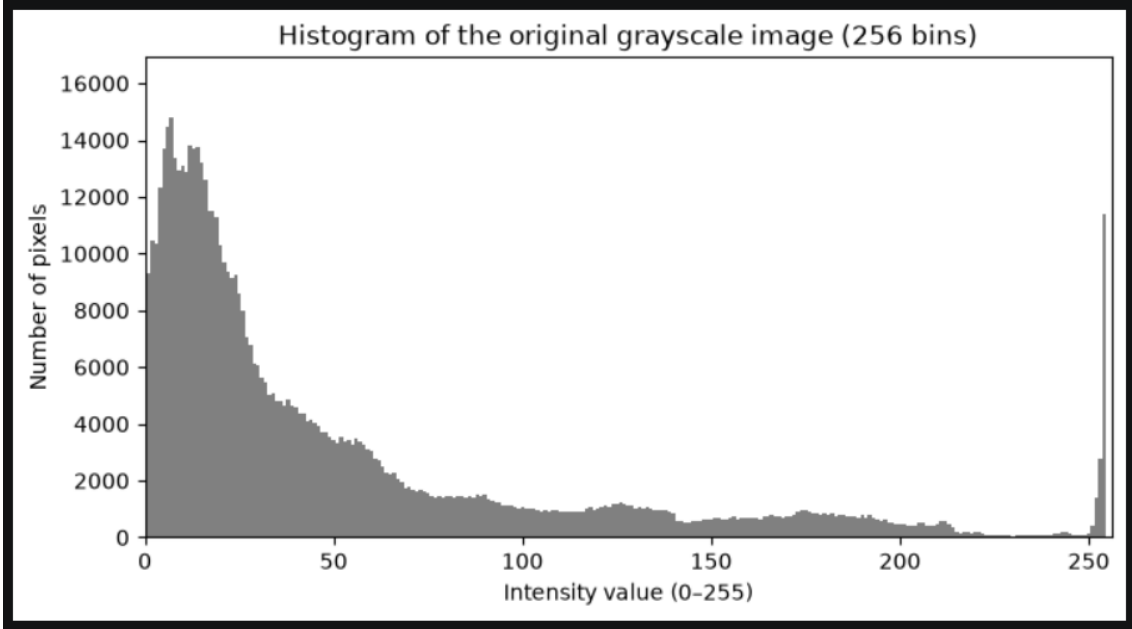
A histogram counts how many pixels have each intensity, so it throws away the positions. For example, the arrangement [[255, 200, 150, 100], [100, 50, 50, 0]] has exactly the same histogram as A but is a different image.

Notebook output



```
hist = cv2.calcHist([original], [0], None, [256], [0, 256]).ravel()
plt.figure(figsize=(8, 4))
plt.bar(np.arange(256), hist, width=1.0, color="gray")
plt.xlim(0, 256)
plt.title("Histogram of the original grayscale image (256 bins)")
plt.xlabel("Intensity value (0–255)"); plt.ylabel("Number of pixels")
plt.show()
print("sum of histogram =", int(hist.sum()), "= number of pixels", original.size)
```

✓ 1.1s




```
sum of histogram = 640000 = number of pixels 640000
```

Observation: Describe where the peaks of our histogram are, and whether the image is dark, bright, low-contrast or well spread (two sentences).

Dry Run 7: Global histogram equalization

r	Freq	Cumulative	CDF	255×CDF	Rounded	255×(CDF−CDFmin)/(1−CDFmin)	Rounded
50	2	2	2/4 = 0.5	127.5	128	0	0
100	1	3	3/4 = 0.75	191.25	191	127.5	128
150	1	4	4/4 = 1	255	255	255	255

CDFmin = 0.5 (the lowest nonzero CDF, at intensity 50). Rounding is to the nearest integer with .5 rounded upward.

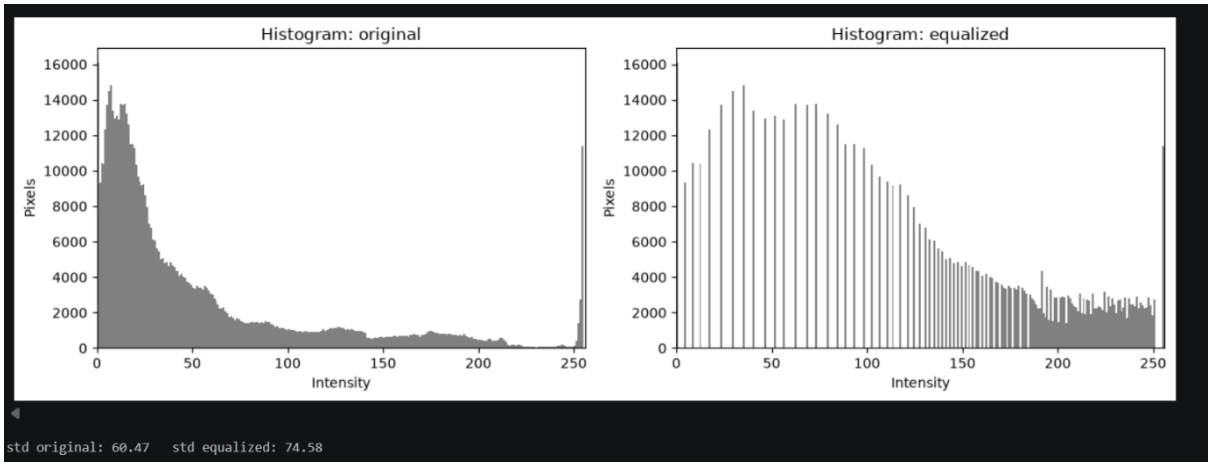
Output list, method 1 (255 × CDF): [128, 128, 191, 255].

Output list, method 2 (255 × (CDF − CDFmin)/(1 − CDFmin)): [0, 0, 128, 255].

OpenCV check: cv2.equalizeHist on [50, 50, 100, 150] returns [0, 0, 128, 255], which matches method 2. OpenCV subtracts the CDF minimum, so the darkest level maps to 0 and the brightest to 255, using the full output range. Method 1 does not map the darkest level to 0.

Notebook output:





Observation: Describe whether detail improved in YOUR image after equalization and whether noise/flat areas became stronger or over-stretched.

Dry Run 8: CLAHE (simplified model)

Bin	Original	After cap 4	+ redistributed	New count	Cumulative	New CDF	255 × CDF
0	6	4	0.5	4.5	4.5	0.5625	143.44
1	2	2	0.5	2.5	7	0.875	223.12
2	0	0	0.5	0.5	7.5	0.9375	239.06
3	0	0	0.5	0.5	8	1	255

Steps: (1) Clip counts above the cap of 4: bin 0 goes from 6 to 4, so the excess is 2. (2) Redistribute the excess equally over 4 bins: 2/4 = 0.5 per bin (fractions allowed). (3) The new histogram is [4.5,

2.5, 0.5, 0.5], still totalling 8. (4) The cumulative counts are [4.5, 7, 7.5, 8], so the new CDF is [0.5625, 0.875, 0.9375, 1.0].

Effect: without clipping, the CDF would be [0.75, 1, 1, 1], so the 6 pixels in bin 0 would grab 75% of the output range. Clipping flattens the steepest part of the CDF, which limits contrast amplification (and noise amplification) in that region.

Interpolation: CLAHE computes a separate mapping for every tile. A pixel near the border between two tiles would get a visibly different output depending on which tile's mapping was used, causing blocky seams. So each pixel is mapped using the mappings of the nearest tile centres and the results are blended by bilinear interpolation.

Note: this simplified model differs from OpenCV, which redistributes integer counts, and the absolute cap of 4 here is not the same thing as the clipLimit parameter (which is a multiple of the average bin count).

Notebook output:

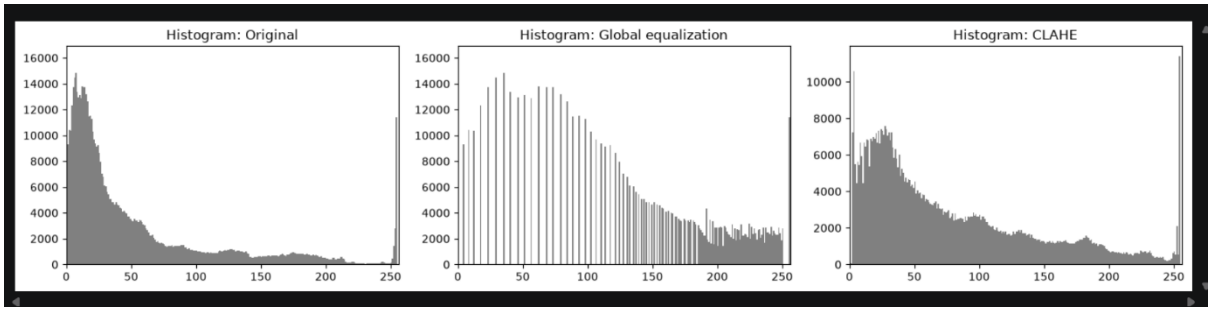
Dry run 8 — histogram [6, 2, 0, 0], absolute bin cap 4

```
H = [6, 2, 0, 0]
cap = 4
clipped = [min(h, cap) for h in H]
excess = sum(H) - sum(clipped)
add = Fraction(excess, len(H))
newH = [Fraction(c) + add for c in clipped]
print("clipped          :", clipped, "| excess:", excess)
print("add per bin      :", add)
print("new histogram    :", [float(x) for x in newH], "| total:", float(sum(newH)))
cs = np.cumsum([float(x) for x in newH])
cdf = cs / cs[-1]
print("cumulative       :", cs.tolist())
print("new CDF          :", cdf.tolist())
print("255*CDF          :", (255 * cdf).round(2).tolist())
assert cdf.tolist() == [0.5625, 0.875, 0.9375, 1.0]
```

✓ 0.0s

```
clipped          : [4, 2, 0, 0] | excess: 2
add per bin      : 1/2
new histogram    : [4.5, 2.5, 0.5, 0.5] | total: 8.0
cumulative       : [4.5, 7.0, 7.5, 8.0]
new CDF          : [0.5625, 0.875, 0.9375, 1.0]
255*CDF          : [143.44, 223.12, 239.06, 255.0]
```





Observation: Compare CLAHE with global equalization on our image, especially in unevenly lit areas. Note that (8, 8) is the number of tiles, not pixels per tile.

Dry Run 9: Fixed binary thresholding

r	Test (T = 127)	Output	Test (T = 180)	Output
0	0 > 127 → false	0	0 > 180 → false	0
80	80 > 127 → false	0	80 > 180 → false	0
127	127 > 127 → false	0	127 > 180 → false	0
128	128 > 127 → true	255	128 > 180 → false	0
180	180 > 127 → true	255	180 > 180 → false	0
255	255 > 127 → true	255	255 > 180 → true	255

White-pixel count: T = 127 gives 3 white pixels (128, 180, 255); T = 180 gives 1 white pixel (255 only). The value 127 is not white for T = 127 because the rule is strictly r > T, and 180 is not white for T = 180 for the same reason. Raising T makes the output darker and keeps fewer pixels.

Notebook output:

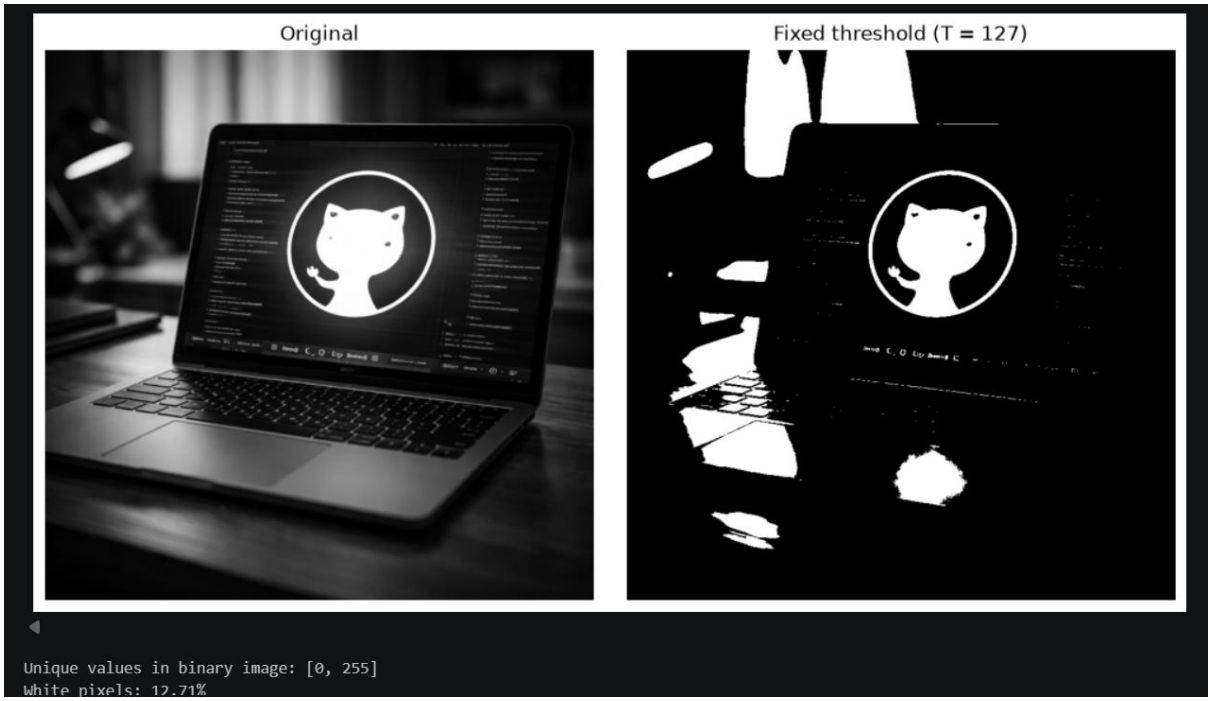
9. Fixed binary thresholding (T = 127)

Dry run 9 — pixels [0, 80, 127, 128, 180, 255]

```
T9 = np.array([0, 80, 127, 128, 180, 255])
for T in (127, 180):
    out = np.where(T9 > T, 255, 0)
    print(f"T={T}: {out.tolist()}  white pixels: {np.count_nonzero(out == 255)}")
assert np.where(T9 > 127, 255, 0).tolist() == [0, 0, 0, 255, 255, 255]
assert np.where(T9 > 180, 255, 0).tolist() == [0, 0, 0, 0, 0, 255]
```

✓ 0.0s

```
T=127: [0, 0, 0, 255, 255, 255]  white pixels: 3
T=180: [0, 0, 0, 0, 0, 255]  white pixels: 1
```



Observation:

The binary image contains only the unique values 0 and 255. State the percentage of white pixels for YOUR image.

Dry Run 10: Otsu thresholding

T	Class 0 (r ≤ T)	Class 1 (r > T)	w0	w1	μ0	μ1	$w0 \cdot w1 \cdot (\mu0-\mu1)^2$
0	{0}	{1, 4, 5}	0.25	0.75	0	3.3333	2.0833
1	{0, 1}	{4, 5}	0.5	0.5	0.5	4.5	4
2	{0, 1}	{4, 5}	0.5	0.5	0.5	4.5	4
3	{0, 1}	{4, 5}	0.5	0.5	0.5	4.5	4
4	{0, 1, 4}	{5}	0.75	0.25	1.6667	5	2.0833

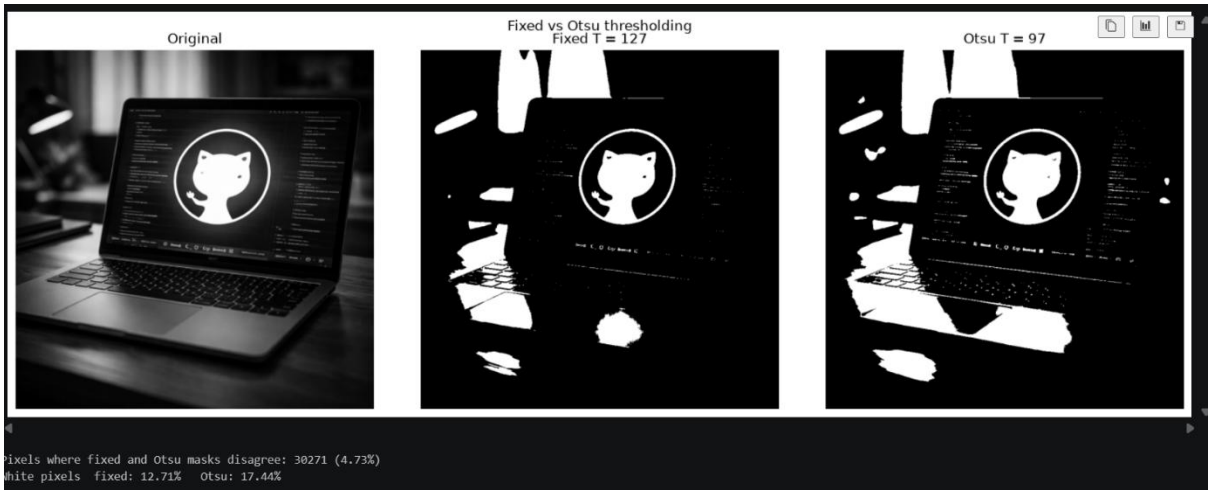
The maximum between-class variance is 4, tied at T = 1, 2 and 3 (no pixel has value 2 or 3, so those splits are identical). Choosing the smallest tied T gives T = 1.

Predicted mask (r > 1): [0, 0, 255, 255], i.e. pixels 0 and 1 become 0, and pixels 4 and 5 become 255.

OpenCV check: cv2.threshold(..., 0, 255, THRESH_BINARY + THRESH_OTSU) returns threshold 1 and mask [0, 0, 255, 255], matching the prediction.

Notebook output:

```
T=0: class0=[0] class1=[1, 4, 5] w0=0.25 w1=0.75 mu0=0.0000 mu1=3.3333  w0*w1*(mu0-mu1)^2=2.0833
T=1: class0=[0, 1] class1=[4, 5] w0=0.50 w1=0.50 mu0=0.5000 mu1=4.5000  w0*w1*(mu0-mu1)^2=4.0000
T=2: class0=[0, 1] class1=[4, 5] w0=0.50 w1=0.50 mu0=0.5000 mu1=4.5000  w0*w1*(mu0-mu1)^2=4.0000
T=3: class0=[0, 1] class1=[4, 5] w0=0.50 w1=0.50 mu0=0.5000 mu1=4.5000  w0*w1*(mu0-mu1)^2=4.0000
T=4: class0=[0, 1, 4] class1=[5] w0=0.75 w1=0.25 mu0=1.6667 mu1=5.0000  w0*w1*(mu0-mu1)^2=2.0833
Chosen T (smallest among tied maxima): 1
Predicted mask: [0, 0, 255, 255]
OpenCV threshold: 1.0 | OpenCV mask: [0, 0, 255, 255] | matches prediction: True
```



Observation: State the threshold returned by cv2.THRESH_OTSU for YOUR image and how many pixels differ from the T = 127 mask.

Dry Run 11: Adaptive Gaussian thresholding

Position	Value	Weight	Value × weight numerator	Contribution
(0,0)	100	1/16	100	6.25
(0,1)	100	2/16	200	12.5
(0,2)	100	1/16	100	6.25
(1,0)	100	2/16	200	12.5
(1,1)	120	4/16	480	30
(1,2)	140	2/16	280	17.5
(2,0)	140	1/16	140	8.75
(2,1)	140	2/16	280	17.5
(2,2)	140	1/16	140	8.75

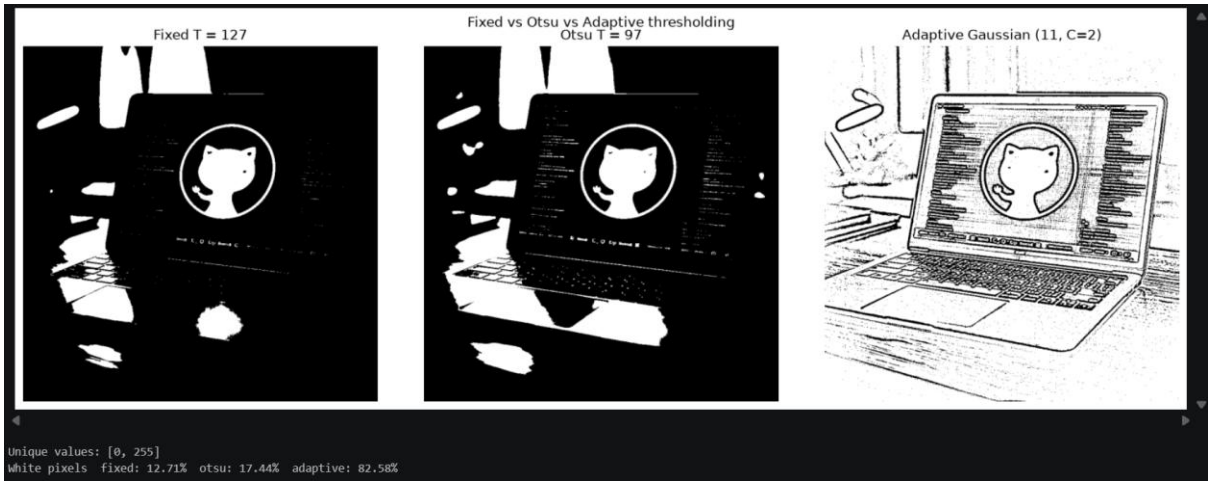
Weighted sum = 1920/16 = 120. Threshold T = mean − C = 120 − 2 = 118.

Center pixel = 120. Test 120 > 118 is true, so the output is 255 (white). A pixel exactly equal to T (118) fails the test 118 > 118, so it becomes 0 (black), because the rule is strictly greater than.

In the image task the same idea is applied with an 11 × 11 Gaussian-weighted neighbourhood, so every pixel has its own local threshold. This helps when illumination is uneven (shadows, gradients), where a single global T would turn one whole region white or black.

Notebook output:

```
weighted mean: 120.0 | T = mean - C = 118.0
center 120.0 > 118.0  ->  255 (white)
pixel equal to T (118.0) > T  ->  0 (black, rule is strictly greater)
```



Observation: Compare adaptive Gaussian (blockSize 11, C = 2) with the fixed and Otsu masks on YOUR image, and say when a local threshold helps (uneven illumination).

4. Concepts

Clipping: Clipping forces values outside the valid range to the nearest limit (below 0 becomes 0, above 255 becomes 255). It prevents wrap-around, but different input values become the same output value, so that detail is permanently lost (e.g. 215 and 240 both become 255 after +40).

Overflow: An 8-bit unsigned integer can only hold 0–255 and wraps modulo 256 (240 + 40 = 24, 30 − 40 = 246), which turns bright pixels dark and vice versa. To avoid it, the arithmetic is done in int32 or float64, the result is clipped to 0–255, and only then converted to uint8.

Brightness versus contrast: Brightness adds a constant β to every pixel and shifts the histogram without changing its spread. Contrast multiplies by α and changes the spread: $\alpha > 1$ expands, $\alpha < 1$ compresses. Multiplying around 128, $s = (r - 128)\alpha + 128$, changes contrast while leaving mid-gray unchanged; $s = \alpha r$ also changes the overall brightness.

Gamma convention: Here $s = 255 \cdot (r/255)^{\gamma}$. Values are normalised to 0–1 first, so 0 and 1 stay fixed. $\gamma < 1$ brightens (expands dark values), $\gamma > 1$ darkens (compresses dark values). The opposite convention, $s = 255 \cdot (r/255)^{(1/\gamma)}$, swaps these directions, so the formula in use must always be stated.

Global versus local enhancement: Global histogram equalization and a fixed or Otsu threshold use one mapping for the entire image: they are fast and work well when the lighting is even, but fail with shadows or gradients and can amplify noise in flat regions. Local methods (CLAHE, adaptive thresholding) compute a mapping or threshold per neighbourhood, which handles uneven illumination and reveals local detail; CLAHE limits noise amplification with the clip limit, while adaptive thresholding can still produce noise in flat areas.